REDES DE SENSORES E ATUADORES

(TAREFA 1 DA UNIDADE 3, CAPÍTULO 1)

Aluno: Alan Sovano Gomes

Matrícula: 202420110940138

Trilha: Software Embarcado

Mentora: Gabriela Teixeira

1. Enunciados

- **Questão (1)** – *Leitura de Luminosidade com o Sensor BH1750*: configurar o sensor BH1750 para medir a intensidade de luz, utilizando o RP2040. A programação do microcontrolador será feita para coletar os dados de luminosidade e exibi-los diretamente em um monitor serial, permitindo a observação em tempo real das variações na intensidade da luz. Além disso, como um desafio extra, programe o RP2040 para ajustar automaticamente a posição de um Servo Motor 9G SG90 em resposta ao nível de luz captado pelo sensor, de forma que o servo reaja dinamicamente conforme a iluminação detectada

- **Questão (2)** – *Monitoramento de Temperatura e Umidade com o Sensor AHT10:* configurar o sensor AHT10 para monitorar tanto a temperatura quanto a umidade do ambiente, utilizando o RP2040. A programação do microcontrolador deverá coletar e exibir esses dados em uma tela LCD 320x240, possibilitando uma visualização em tempo real. Como um desafio adicional, ajuste o código para que o LCD exiba um aviso caso a umidade ultrapasse 70% ou a temperatura fique abaixo de 20°C, criando uma interface que alerta o usuário sobre condições específicas do ambiente.

- **Questão (3)** – *Controle e Monitoramento de Movimento com MPU6050:* configurar o sensor MPU6050 para captar dados de movimento, como aceleração e rotação, e exibir as leituras de inclinação no monitor serial. Além disso, programe o RP2040 para ajustar a posição de um Servo Motor 9G SG90 em função do ângulo de inclinação detectado, promovendo um controle dinâmico de movimento em resposta à orientação do sensor. No desafio extra, você deverá adicionar um alerta visual na tela LCD

320x240 que indique quando o sistema ultrapassa um determinado ângulo de inclinação, criando um sistema de monitoramento visual das mudanças de posição.

OBS: Devido a falta do GPS e do módulo LoRa no kit de sensores recebidos no programa de residência, essas questões não foram resolvidas. Para fins deste relatório, a Questão 04 do comando original (referente ao sensor MPU 6050) foi renomeada como "Questão 03".

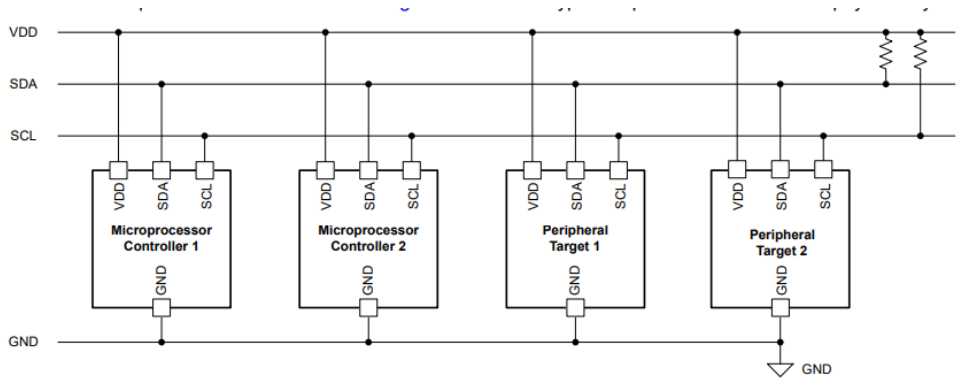
2. Considerações iniciais

Para a solução dos problemas propostos, foi elaborado um código em linguagem C para cada enunciado de cada questão, totalizando quatro códigos desenvolvidos e testados na BitDogLab. Devido à divergência entre os comandos das questões e os dispositivos recebidos para as atividades práticas, algumas alterações foram feitas na hora de implementar as solução pedidas, dentre elas:

- No lugar da tela LCD 320x240, foi utilizado o *display* OLED já disponível na placa, o qual funciona com o protocolo de comunicação I2C (*Inter-Integrated Circuit*);
- O servo motor recebido não é de posição, mas de velocidade (a variação no sinal de PWM do motor faz que a velocidade e o sentido do motor variem). Logo, quando exigido um controle de posição do motor, foi realizado um controle de velocidade;

De forma geral, todas as soluções envolvem a utilização do protocolo de comunicação I2C, o qual é utilizado para realizar a comunicação com os sensores BH1750, AHT10, MPU6050 e com o *display* OLED. No protocolo I2C, é possível conectar vários dispositivos em um mesmo barramento, com todos os dispositivos sujeitos a um mesmo ciclo de *clock*. Cada um dos elementos conectados ao barramento é ativado a partir de um endereço específico e único para, então, ser capaz de enviar ou receber dados (Wu, 2022). A Figura 1 ilustra a conexão de diversos dispositivos em um barramento I2C.

Figura 1 – dispositivos conectados em um barramento que funciona de acordo com o protocolo I2C.



Fonte: Wu (2022).

Ao longo das soluções que serão apresentadas, comentários adicionais são feitos sobre as especificidades de cada projeto – como a utilização do PWM (*Pulse-Width Modulation*) para o controle do servo motor.

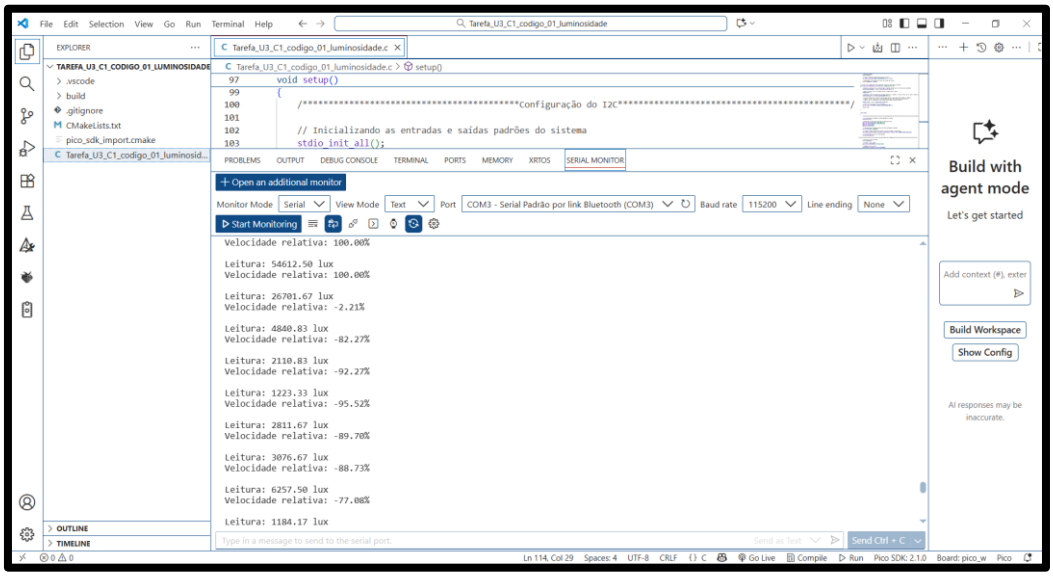
.

3. Solução da Questão 01 (utilização do sensor BH1750)

Para escrever o código da questão 01, foi necessário avaliar sua folha de dados em busca do endereço do dispositivo (0x23) e da sequência de comandos para manipular os registradores internos. Além disso, foi necessário verificar o tempo de amostragem e a resoluções permitidas pelo dispositivo, sendo escolhida uma resolução de 1 lux (referente a um período de amostragem máximo de 120 ms) (ROHM SEMICONDUCTOR, 2011). Para o código em questão, foi realizada uma amostragem a cada 200 ms (80 ms acima do intervalo mínimo permitido).

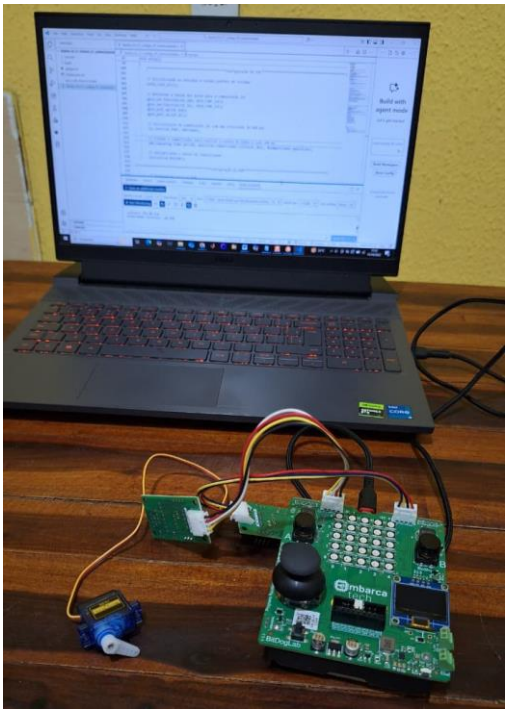
Para a utilização do servo motor, é necessário aplicar um sinal PWM com frequência de 50 Hz (ou com um período de 20 ms). Se a largura do pulso (tempo em que o sinal PWM fica em nível lógico alto) for de 1 ms, então a velocidade do motor será máxima e em um sentido; no caso da largura do pulso ser de 2 ms, a velocidade será máxima no sentido contrário. Dessa forma, no código desenvolvido, foi feita uma conversão entre a luminosidade lida pelo sensor e a velocidade de giro do motor: quando a luminosidade é mínima, ele deve girar totalmente em um sentido; quando a luminosidade é máxima, ele deve gerar totalmente no outro sentido. As Figuras 2 e 3 ilustram, respectivamente, as informações que são exibidas ao longo do uso do sistema, além das conexões feitas no hardware para que o projeto funcionasse. Após os testes, foi possível concluir que o projeto funcionou conforme esperado.

Figura 2 – Valores de luminosidade lidos e mostrados no monitor serial, bem como a velocidade relativa do servo motor.



Fonte: Autor (2025).

Figura 3 – Montagem experimental para validação do projeto.



Fonte: Autor (2025).

4. Solução da Questão 02 (utilização do sensor AHT10)

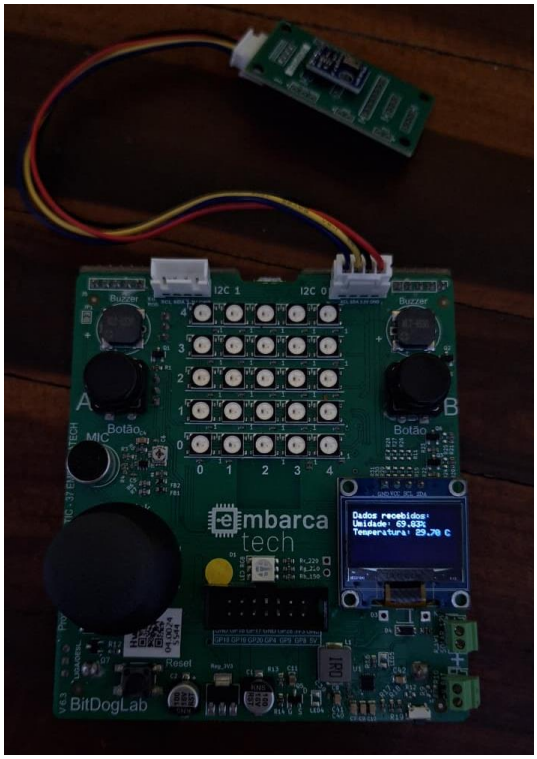
Assim como na Questão 01, para resolver a Questão 02 foi necessário avaliar a folha de dados do sensor AHT10 (o qual possui o endereço 0x38). A informação do sensor é dada

em conjuntos de seis bytes, os quais devem ser corretamente separados e convertidos para as informações de temperatura (em graus Celsius) e de umidade relativa (AOSONG ELECTRONICS, 2018). O sistema foi configurado para que uma leitura do sensor fosse captada a cada um segundo.

Além da utilização do sensor em questão, o *display* OLED da BitDogLab foi utilizado para mostrar os valores de umidade e temperatura lidos. O sensor de temperatura foi conectado ao barramento I2C0 da BitDogLab, enquanto o barramento I2C1 é utilizado internamente para se comunicar com o *display* OLED. O sistema também foi configurado para mostrar avisos no *display* caso a temperatura ficasse abaixo de 20 ºC ou a umidade relativa ficasse acima de 70%.

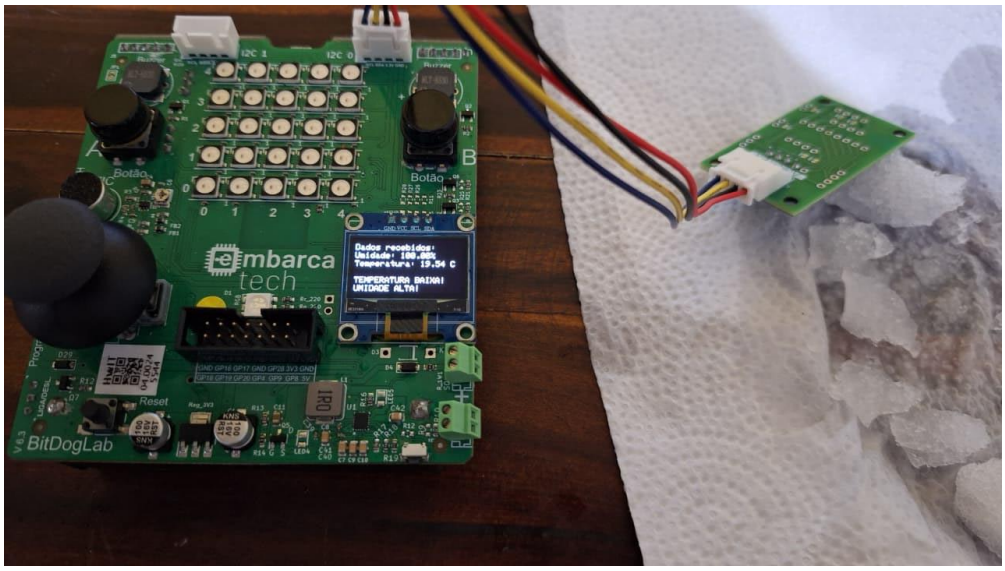
Na Figura 4, é possível visualizar o sistema em uma situação padrão de leitura, medindo as condições do ambiente em que está situado. Na Figura 5, um pouco de gelo derretendo foi colocado próximo ao sistema, de forma a fazer com que houvesse uma elevação na umidade e queda na temperatura. Como é possível visualizar, essa mudança de condições ambientais disparou ambos os avisos do sistema, demonstrando que o mesmo está funcionando conforme desejado.

Figura 4 – Leitura do sistema em condições ambientais típicas.



Fonte: Autor (2025).

Figura 5 – Leitura do sistema em condições criadas para disparar os avisos de temperatura e umidade críticos.



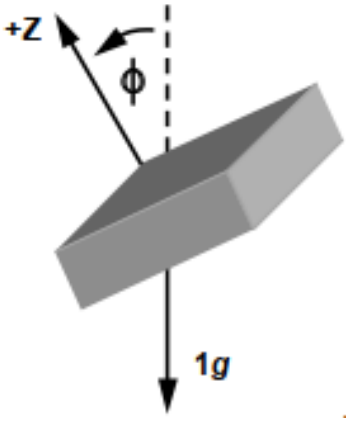
Fonte: Autor (2025).

5. Solução da Questão 03 (utilização do sensor AHT10)

Para a Questão 03, o sensor MPU 6050 foi utilizado. Este sensor é, na verdade, uma "*Motion Processing Unit*" ou Unidade de Processamento de Movimento. Ele disponibiliza dados de um acelerômetro (para medir a aceleração nos três eixos cartesianos) e de um giroscópio (para medir a velocidade angular em torno dos três eixos). O endereço da MPU 6050 é 0x68 e ele possui uma vasta gama de possibilidades de configuração, com cada dado disponibilizado (seja do acelerômetro ou do giroscópio) sendo de 16 bits de tamanho (INVENSENSE, 2013).

No projeto, foi configurado que a captura dos dados dos sensores deveria ocorrer em intervalos periódicos de 50 ms. Além disso, no projeto foi calculado o ângulo de inclinação do plano XY (ou seja, a inclinação com relação ao eixo Z). Esse ângulo pode ser calculado através dos dados do acelerômetro, o qual (em condições de repouso) nos fornece os valores da força da gravidade projetada sobre seus três eixos. A Figura 6 ilustra uma situação onde há a medição do ângulo entre o eixo Z e o plano XY, denominado de ϕ.

Figura 6 – Ilustração do ângulo ϕ a ser estimado (inclinação do acelerômetro)



Fonte: Fisher (2010).

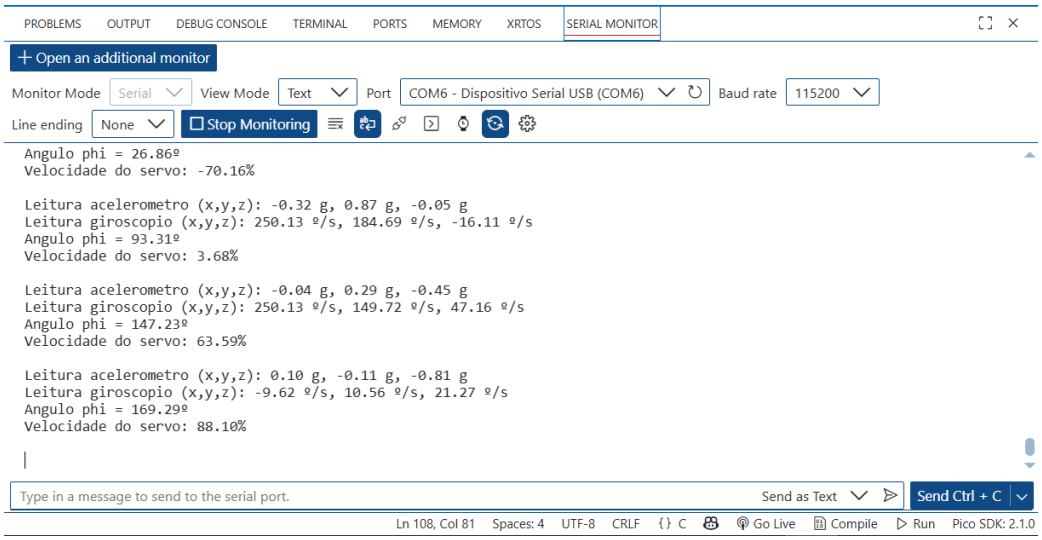
No projeto, de acordo com a inclinação do acelerômetro, a velocidade e o sentido de giro do servo motor é ajustada (de forma similar ao que ocorre no exemplo da Questão 01). Além disso, quando a inclinação do plano é maior que 90º, um aviso é mostrado no *display* OLED (indicando o sistema detectou uma inclinação crítica). Além do aviso, o *display* também mostra qual o ângulo de inclinação atual, calculado pela seguinte fórmula (Fisher, 2010):

$$\phi = \cos^{-1}\left(\frac{A_z}{\sqrt{A_x^2 + A_y^2 + A_z^2}}\right)$$

Onde A_x, A_y e A_z são, respectivamente, os valores da aceleração detectada em cada eixo do acelerômetro.

Por fim, o sistema mostra, através do monitor serial, outros tipos de informações: o valor de cada um dos eixos do acelerômetro em g (múltiplos da aceleração da gravidade), além de cada um dos valores enviados pelo giroscópio (em º/s), do ângulo de inclinação e da velocidade relativa do motor. Na Figura 7, é possível visualizar como as informações são mostradas ao usuário pela porta serial.

Figura 7 – Dados enviados pela porta serial sobre o acelerômetro, o giroscópio, o ângulo de inclinação e a velocidade do motor.



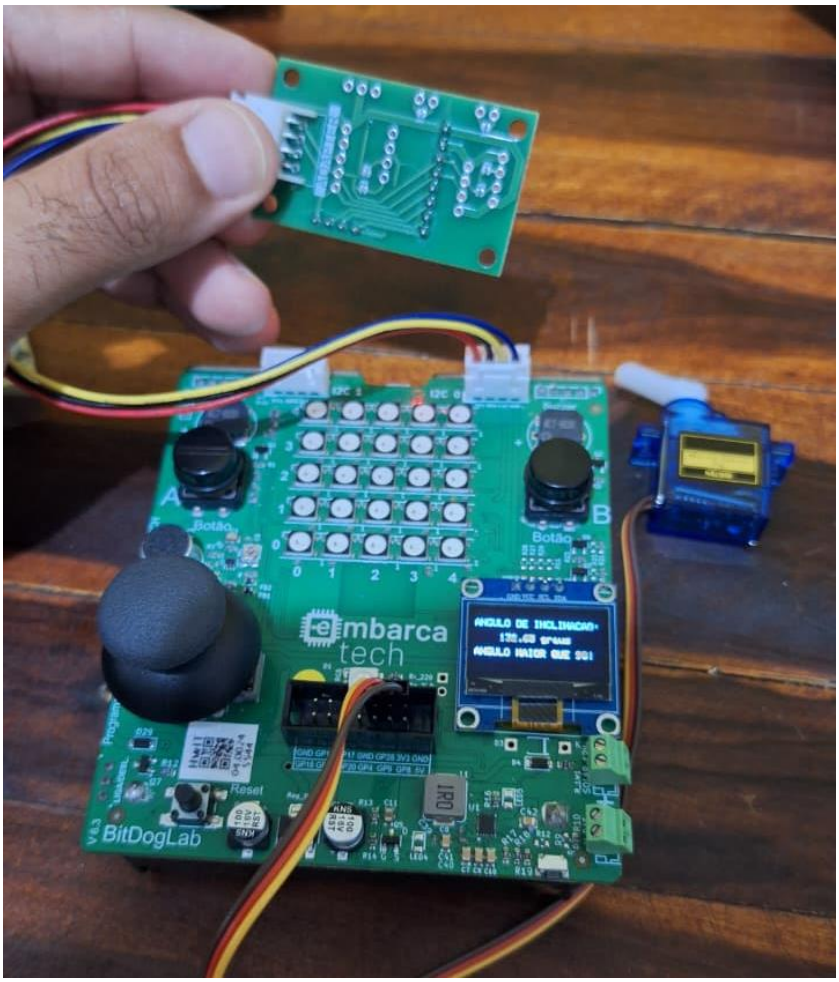
Fonte: Autor (2025).

Nas Figuras 8 e 9, é possível visualizar o sistema em funcionamento, com a Figura 9 demonstrando que o sistema é capaz de detectar quando o ângulo crítico de 90º é alcançado. Dessa forma, é possível inferir que o sistema está funcional dentro das especificações solicitadas.

Figura 8 – Sistema funcionando (sem ter alcançado o ângulo crítico).

Fonte: Autor (2025).

Figura 9 – Sistema funcionando (após ter alcançado o ângulo crítico).



Fonte: Autor (2025).

Diante do exposto, é possível confirmar que todos os códigos em C desenvolvidos foram capazes de realizar os objetivos propostos pelas Questões 1, 2 e 3. Por fim, para comprovar o funcionamento dos códigos, o link a seguir leva para uma pasta no Google Drive que possui os arquivos dos projetos desenvolvidos (arquivos principais e bibliotecas auxiliares), bem como vídeos dos sistemas funcionando na placa BitDogLab.

https://drive.google.com/drive/folders/1NwVw3iMCF0NofvXp7aqSfySsoTsirYbq?usp=sharing

6. Conclusão

A partir do desenvolvimento da atividade, foi possível aprofundar conceitos sobre protocolos de comunicação para interfaceamento com sensores e atuadores. Foi possível perceber que, ao se trabalhar com sensores digitais, é necessário ter conhecimento dos principais protocolos de comunicação (como o I2C), além de ser capaz de realizar uma leitura técnica da folha de dados dos respectivos dispositivos.

De forma geral, foi possível concluir também que a utilização de sensores digitais é muito prática, sendo possível configurar vários dispositivos, de forma semelhante, em um mesmo sistema (como na utilização do *display* OLED e do sensor de temperatura e umidade). Além disso, foi possível verificar como um sensor pode ser utilizado no controle de um atuador, como um servo motor (o que foi o caso na utilização do sensor de luminosidade e na MPU 6050). De forma geral, a confecção do primeiro código foi mais demorada, mas com ele elaborado, foi possível reaproveitá-lo para a execução dos outros projetos.

Por fim, vale ressaltar a importância da utilização de sensores e atuadores em sistemas envolvendo Internet das Coisas, visto que os sensores são a forma dos sistemas computacionais perceberem o mundo, enquanto os atuadores são a forma e o sistema possui para interagir com o ambiente ao seu redor.

7. Código em linguagem C desenvolvido para a placa BitDogLab (questão 01)

```
/*
Embarcatech - Trilha de Software Embarcado - Tarefa da Unidade 3 Capítulo 1
Questão 01 (Sensor de umidade)
Aluno: Alan Sovano Gomes
Matrícula: 202420110940138
Mentora: Gabriela Teixeira
*/

    // Bibliotecas utilizadas
    #include <stdio.h>
    #include "pico/stdlib.h"
    #include "hardware/i2c.h"
    #include "hardware/pwm.h"


    // Definindo os pinos da comunicação I2C (utilizando a I2C0 pelos pinos GP0 e
GP1)
    #define I2C_PORT i2c0
    #define I2C_SDA 0
    #define I2C_SCL 1
    #define BH1750_ADDR 0x23 // Endereço do dispositivo I2C (sensor de
luminosidade)

    //Defines para o PWM (servomotor)
    #define SERVO 2 // Pino ao conectar o servo no encaixe I2C1 da placa

    // Variáveis de configuração do PWM.
    // f_PWM = 125*(10^6)/((DIVIDER)*(PERIOD+1))
    // Se PERIOD = 24999 e DIVIDER = 100 -> f_PWM = 50 Hz
    const uint16_t PERIOD_PWM =24999;
    const float DIVIDER_PWM = 100;

    // Variáveis auxiliares para o controle do servo motor
    float normalizacao_servo = 0.0; // VNormalização do valor de luminosidade para
controle do servo
    float rotacao_servo = 0.75*(PERIOD_PWM+1); // Ponto médio da rotação
(velocidade zero)
    float print_velocidade = 0;


    // Variáveis globlais
    struct repeating_timer temporizador_aquisicao;
    float leitura_luminosidade= 0.0;
    bool flag_printf = 0;

    /************************Funções para operar o sensor*******************/

    void inicializar_bh1750()
```

```
    {
        // 0x01 -> Liga o sensor;
        // 0x07 -> Zerando o conteúdo do registrador de dados após a inicialização
        // 0x10 -> Modo de alta resolução 01 (1 lux de resolução com amostragem a
cada 120 ms);
        uint8_t buf[3] = {0x01, 0x07, 0x10};

        for(int i =0; i<3; i++)
        {
            i2c_write_blocking(I2C_PORT, BH1750_ADDR, &buf[i], 1, false);
        }

    }

    // Função para ler e converter os dados que chegam do sensor de luminosidade
via I2C
    uint16_t capta_medicao_bh1750()
    {
        // Buffer de dados
        uint8_t buf[2];

        // Lendo a informação disponibilizada pelo sensor
        i2c_read_blocking(I2C_PORT, BH1750_ADDR, buf, 2, false);

        // Combinando os dois dados de 8 bits em um dado de 16 bits
        return (buf[0]<<8 | buf[1]);
    }

    // Função a ser chamada de forma repetitiva (aquisição periódica dos dados)
    bool aquisicao_temporizador_callback(struct repeating_timer *t)
    {
        // Realiza a leitura e faz a convrsão para float de acordo com as
orientações do datasheet
        leitura_luminosidade = (float) capta_medicao_bh1750()/1.2;

        // Após a leitura, seta o flag para enviar os dados para a serial
        flag_printf = 1;

        // Converte o valor do sensor para um valor entre 0 e 1 (54612.5 é valor
máximo, em lux, que o sensor alcança)
        normalizacao_servo = (leitura_luminosidade/54612.5);

        // O valor passado para a função "pwm_set_gpio_level()" deve estar entre
1000 ms e 2000 ms
        // 1000 ms -> Faz o servo girar ao máximo para um lado (5% de duty cycle
para 50 Hz)
        // 2000 -> gira o máximo para o outro lado (10% de duty cycle para 50 Hz)

        rotacao_servo = 0.05 + normalizacao_servo*0.05;
```

```
        // Usa o valor do duty_cycle no servomotor
        pwm_set_gpio_level(SERVO, rotacao_servo*(PERIOD_PWM+1));

        return true;

    }


    void setup()

    {
        /****************************************Configuração do
I2C*********************************************/

        // Inicializando as entradas e saídas padrões do sistema
        stdio_init_all();

        // Definindo a função dos pinos para a comunicação I2C
        gpio_set_function(I2C_SDA, GPIO_FUNC_I2C);
        gpio_set_function(I2C_SCL, GPIO_FUNC_I2C);
        gpio_pull_up(I2C_SDA);
        gpio_pull_up(I2C_SCL);

        // Inicialização da comunicação I2C com uma velocidade de 400 kHz
        i2c_init(I2C_PORT, 400*1000);

        // Criando o temporizador para realizar a coleta de dados a cada 200 ms
        add_repeating_timer_ms(200, aquisicao_temporizador_callback, NULL,
&temporizador_aquisicao);

        // Inicializando o sensor de luminosidade
        inicializar_bh1750();

/****************************************Configuração do
PWM*********************************************/

        // Variável de slice do PWM
        uint slice_servo;

        // Setando o pino de PWM
        gpio_set_function(SERVO, GPIO_FUNC_PWM);

        // Obtendo valor do slice
        slice_servo = pwm_gpio_to_slice_num(SERVO);

        // Definindo divisor de clock do PWM
        pwm_set_clkdiv(slice_servo, DIVIDER_PWM);

        // Valor do contador do PWM (período)
        pwm_set_wrap(slice_servo, PERIOD_PWM);
```

```
        // Valor inicial do pino PWM
        pwm_set_gpio_level(SERVO, 0.075*(PERIOD_PWM+1));

        // Habilitando o PWM no slice correspondente
        pwm_set_enabled(slice_servo, true);


    }



    int main()
    {
        // Realizando as configurações iniciais
        setup();

        // Verificação periódica para a realização do printf
        while (true)
        {
            // Enviando o valor lido para o monitor serial
            if(flag_printf==1)
            {
                // Valor de -100% a 100% da velocidade (o sinal indica a rotação
para um lado ou para o outro)
                print_velocidade = -100 + 200*normalizacao_servo ;

                printf("Leitura: %.2f lux\nVelocidade relativa: %.2f%%\n\n",
leitura_luminosidade, print_velocidade);
                flag_printf = 0;
            }

        }

        return 0;
    }
```

8. Código em linguagem C desenvolvido para a placa BitDogLab (questão 02)

```c
/*
Embarcatech - Trilha de Software Embarcado - Tarefa da Unidade 3 Capítulo 1
Questão 02 (Sensor de temperatura e umidade)
Aluno: Alan Sovano Gomes
Matrícula: 202420110940138
Mentora: Gabriela Teixeira
*/
    // Bibliotecas utilizadas
    #include <stdio.h>
    #include "pico/stdlib.h"
    #include "hardware/i2c.h"
    #include "hardware/pwm.h"

    // Biblioteca utilizada para trabalhar com o display OLED
    #include "ssd1306.h"

    // Definindo os pinos da comunicação I2C (utilizando a I2C0 pelos pinos GP0 e
GP1)
    #define I2C_PORT i2c0
    #define I2C_SDA 0
    #define I2C_SCL 1
    #define AHT10_ADDR 0x38 // Endereço do dispositivo I2C (temperatura e umidade)

    // Limites para avisos no display OLED (MUDAR AQUI PARA DEPURAÇÃO E TESTES DA
INTERFACE!)
    #define LIMIAR_TEMP 20
    #define LIMIAR_UMIDADE 70

    // Variáveis globlais
    struct repeating_timer temporizador_aquisicao;
    float leitura_temp = 0.0;
    float leitura_umidade = 0.0;
    bool flag_printf = 0;
    const uint8_t comando_sensor[3] = {0xAC, 0x33, 0x00}; // Sequência de
inicialização da leitura
    const char *words[]= {"TEMPERATURA BAIXA!", "UMIDADE ALTA!","Dados recebidos:
"}; // Strings utilizadas no display OLED

    // Guardam os dados que serão mostrados no display OLED
    char info [20];
    char info2 [25];


    /************************Funções para operar o sensor******************/

    void inicializar_aht10()
    {
        // 0xE1 -> Inicializa o sensor;
```

```
        // 0x08 -> Comando que envolve a calibração do sensor;
        // 0x00 -> Byte extra contendo zeros (padding)
        uint8_t buf[3] = {0xE1, 0x08, 0x00};

        // Enviando os bytes pelo barramento I2C
        i2c_write_blocking(I2C_PORT, AHT10_ADDR, buf, 3, false);

    }

    // Função para ler e converter os dados que chegam do sensor de temperatura e
umidade via I2C
    // Recebe os endereços de duas variáveis (facilita o envio de várias infos
pela função)
    void capta_medicao_aht10(float *temperatura, float *umidade)
    {
        // Buffer de dados
        uint8_t buf[6];

        // Enviando o comando para leitura (true para manter o controle do
barramento)
        i2c_write_blocking(I2C_PORT, AHT10_ADDR, comando_sensor, 3, true);

        // Lendo os dados (false para encerrar a comunicação com o barramento)]
        // OBS: diferente do sensor de luminosidade, esse não tem modo de
amostragem automática!
        do
        {
           i2c_read_blocking(I2C_PORT, AHT10_ADDR, buf, 6, false);

        } while ( (buf[0]>>7) == 1 ); // Verificando se o bit de estado indica que
a operação de envio de dados por parte do sensor acabou




        // Reconstruindo os dados de temperatura (infos do 2º, 3º e metade do 4º
byte que foram recebidos)
        // OBS: os bytes estão sendo reescritos como infos de 32 bits, por isso o
casting das variáveis (a info tem 20 bits)
        *umidade = 100.0*( (uint32_t) buf[1] <<12 | (uint32_t) buf[2] <<4 |
(uint32_t) buf[3] >> 4 )/1048576;

        // Reconstruindo os dados de temperatura (infos do 6º, 5º e metade do 4º
byte que foram recebidos)
        *temperatura = 200.0*(( ((uint32_t) buf[3] & 0x0F) <<16 ) | (uint32_t)
buf[4] <<8 | (uint32_t) buf[5])/1048576 - 50;

    }
```

```
    // Função a ser chamada de forma repetitiva (aquisição periódica dos dados)
    bool aquisicao_temporizador_callback(struct repeating_timer *t)
    {
        // Realiza a leitura e armazena nas variáveis globais
        capta_medicao_aht10(&leitura_temp, &leitura_umidade);

        // Após a leitura, seta o flag para enviar os dados para a serial
        flag_printf = 1;

        return true;

    }


    void setup()

    {
        /*****************************************Configuração do
I2C*********************************************/

        // Inicializando as entradas e saídas padrões do sistema
        stdio_init_all();

        // Definindo a função dos pinos para a comunicação I2C
        gpio_set_function(I2C_SDA, GPIO_FUNC_I2C);
        gpio_set_function(I2C_SCL, GPIO_FUNC_I2C);
        gpio_pull_up(I2C_SDA);
        gpio_pull_up(I2C_SCL);

        // Inicialização da comunicação I2C com uma velocidade de 400 kHz
        i2c_init(I2C_PORT, 400*1000);


        // Criando o temporizador para realizar a coleta de dados a cada 1000 ms
        add_repeating_timer_ms(1000, aquisicao_temporizador_callback, NULL,
&temporizador_aquisicao);

        // Inicializando o sensor de luminosidade
        inicializar_aht10();

/*****************************************Configuração do
OLED*********************************************/

        //Comunicação I2C do OLED
        i2c_init(i2c1, 400000);
        gpio_set_function(14, GPIO_FUNC_I2C);
        gpio_set_function(15, GPIO_FUNC_I2C);
        gpio_pull_up(14);
        gpio_pull_up(15);
```

```
        // Inicializa o display para desligá-lo (caso estivesse ligado
anteriormente)
        ssd1306_t disp;
        disp.external_vcc=false;
        ssd1306_init(&disp, 128, 64, 0x3C, i2c1);
        ssd1306_poweroff(&disp);

    }


    int main()
    {
        // Realizando as configurações iniciais
        setup();

        // Declarando variável utilizada para definir o display OLED
        ssd1306_t disp;
        disp.external_vcc=false;
        ssd1306_init(&disp, 128, 64, 0x3C, i2c1);
        ssd1306_clear(&disp);


        // Verificação periódica para a realização do printf
        while (true)
        {
            // Enviando o valor lido para o monitor serial
            if(flag_printf==1)
            {

                // Mostrando no monitor serial e guardando as infos em strings
                printf("Temperatura: %.2f C\nUmidade relativa: %.2f%%\n\n",
leitura_temp, leitura_umidade);
                sprintf(info, "Umidade: %.2f%%", leitura_umidade);
                sprintf(info2, "Temperatura: %.2f C", leitura_temp);


                // Escrevendo a mensagem desejada no display OLED:
                ssd1306_draw_string(&disp, 0, 5, 1, words[2]);
                ssd1306_draw_string(&disp, 0, 15, 1, info);
                ssd1306_draw_string(&disp, 0, 25, 1, info2);

                // Checando se a temperatura está abaixo do limiar crítico
                if(leitura_temp<LIMIAR_TEMP)
                {
                    ssd1306_draw_string(&disp, 0, 45, 1, words[0]);
                }

                // Checando se a temperatura está acima do limiar crítico
                if(leitura_umidade>LIMIAR_UMIDADE)
                {
```

```
                ssd1306_draw_string(&disp, 0, 55, 1, words[1]);
            }

            ssd1306_show(&disp);
            ssd1306_clear(&disp);

            flag_printf = 0;
        }

    }

    return 0;
}
```

9. Código em linguagem C desenvolvido para a placa BitDogLab (questão 03)

```c
/*
Embarcatech - Trilha de Software Embarcado - Tarefa da Unidade 3 Capítulo 1
Questão 04 (sensor MPU 6050)
Aluno: Alan Sovano Gomes
Matrícula: 202420110940138
Mentora: Gabriela Teixeira
*/
    // Bibliotecas utilizadas
    #include <stdio.h>
    #include "pico/stdlib.h"
    #include "hardware/i2c.h"
    #include "hardware/pwm.h"
    #include <math.h>

    // Biblioteca utilizada para trabalhar com o display OLED
    #include "ssd1306.h"


    // Definindo os pinos da comunicação I2C (utilizando a I2C1 pelos pinos GP2 e
GP3)
    #define I2C_PORT i2c0
    #define I2C_SDA 0
    #define I2C_SCL 1
    #define MPU6050_ADDR 0x68 // Endereço do dispositivo I2C (MPU 6050)

    //Defines para o PWM (servomotor)
    #define SERVO 28 // Pino ao conectar o servo no canto superior direito do
conector preto da placa

    // Variáveis de configuração do PWM.
    // f_PWM = 125*(10^6)/((DIVIDER)*(PERIOD+1))
    // Se PERIOD = 24999 e DIVIDER = 100 -> f_PWM = 50 Hz
    const uint16_t PERIOD_PWM =24999;
    const float DIVIDER_PWM = 100;
    float normalizacao_servo = 0.0;
    float rotacao_servo = 0.75*(PERIOD_PWM+1);
    float print_velocidade = 0;


    // Variáveis globlais
    struct repeating_timer temporizador_aquisicao;
    bool flag_printf = 0;
    float leitura_ac_x = 0.0, leitura_ac_y = 0.0, leitura_ac_z = 0.0;
    float leitura_gyro_x = 0.0, leitura_gyro_y = 0.0, leitura_gyro_z = 0.0;
    float calc_phi = 0.0;
    const char *words[]= {"ANGULO DE INCLINACAO:", "ANGULO MAIOR QUE 90!"}; //
Strings utilizadas no display OLED
```

```
    // Guardam os dados que serão mostrados no display OLED
    char info [20];


    /************************Funções para operar o sensor******************/

    void inicializar_mpu6050()
    {
        // Resetando o dispositivo
        uint8_t buf[2] = {0x6B, 0x80};

        for(int i =0; i<1; i++)
        {
            i2c_write_blocking(I2C_PORT, MPU6050_ADDR, &buf[i], 1, false);
        }
        sleep_ms(100); // Espera o dispositivo ser estabilizado após o reset


        // Tirando o dispositivo do modo "sleep"
        buf[1] = 0x00;

        for(int i =0; i<1; i++)
        {
            i2c_write_blocking(I2C_PORT, MPU6050_ADDR, &buf[i], 1, false);
        }
        sleep_ms(10); // Espera a tensão dos pinos do dispositivo estabilizar

    }


    // Função para ler e converter os dados que chegam da MPU 6050 via I2C
    void capta_medicao_mpu6050(float *ac_x, float *ac_y, float *ac_z, float
*gyro_x, float *gyro_y, float *gyro_z, float *phi)
    {

        // Buffer de dados (acelerômetro e giroscopio)
        uint8_t buf_ac[6];
        uint8_t buf_gyro[6];

        // Endereços dos registradores iniciais (ocorre um autoincremento após a
leitura)
        uint8_t reg_ac_end = 0x3B;
        uint8_t reg_gyro_end = 0x43;

        // Capturando a informação do acelerômetro (16 bits em complemento de 2)
        i2c_write_blocking(I2C_PORT, MPU6050_ADDR, &reg_ac_end, 1, true);
        i2c_read_blocking(I2C_PORT, MPU6050_ADDR, buf_ac, 6, false);

        // Capturando a informação do giroscópio
        i2c_write_blocking(I2C_PORT, MPU6050_ADDR, &reg_gyro_end, 1, true);
```

```
        i2c_read_blocking(I2C_PORT, MPU6050_ADDR, buf_gyro, 6, false);

        // Conversão dos dados do acelerômetro para múltiplos de "g" (aceleração
da gravidade)
        // O fator de sensibilidade é 16384 (valor padrão)
        *ac_x = (float) ( (int16_t) (buf_ac[0]<<8 | buf_ac[1]) )/16384; //
buf_ac[0] e buf_ac[1]
        *ac_y = (float) ( (int16_t) (buf_ac[2]<<8 | buf_ac[3]) )/16384; //
buf_ac[2] e buf_ac[3]
        *ac_z = (float) ( (int16_t) (buf_ac[4]<<8 | buf_ac[5]) )/16384; //
buf_ac[4] e buf_ac[5]

        // Conversão dos dados do giroscópio para múltiplos de º/s (graus por
segundo)
        // O fator de sensibilidade é 131 (valor padrão)
        *gyro_x = (float) ( (int16_t) (buf_gyro[0]<<8 | buf_gyro[1]) )/131; //
buf_gyro[0] e buf_gyro[1]
        *gyro_y = (float) ( (int16_t) (buf_gyro[2]<<8 | buf_gyro[3]) )/131; //
buf_gyro[2] e buf_gyro[3]
        *gyro_z = (float) ( (int16_t) (buf_gyro[4]<<8 | buf_gyro[5]) )/131; //
buf_gyro[4] e buf_gyro[5]

        // Calcula o ângulo de inclinação, convertendo de radianos para segundos
(phi é a incliniação a partir do eixo z)
        *phi = 57.29*acos(*ac_z/sqrt(pow(*ac_x,2) + pow(*ac_y,2) + pow(*ac_z,2)));


    }

    // Função a ser chamada de forma repetitiva (aquisição periódica dos dados)
    bool aquisicao_temporizador_callback(struct repeating_timer *t)
    {
        // Realiza a leitura e faz a convrsão para float de acordo com as
orientações do datasheet
        capta_medicao_mpu6050(&leitura_ac_x, &leitura_ac_y, &leitura_ac_z,
&leitura_gyro_x, &leitura_gyro_y, &leitura_gyro_z, &calc_phi);

        // Após a leitura, seta o flag para enviar os dados para a serial
        flag_printf = 1;

        // Converte o valor do sensor para um valor entre 0 e 1
        normalizacao_servo = calc_phi/180;

        // O valor passado para a função "pwm_set_gpio_level()" deve estar entre
1000 ms e 2000 ms
        // 1000 ms -> Faz o servo girar ao máximo para um lado (5% de duty cycle
para 50 Hz)
        // 2000 ms -> gira o máximo para o outro lado (10% de duty cycle para 50
Hz)
```

```
        rotacao_servo = 0.05 + normalizacao_servo*0.05;

        // Usa o valor do duty_cycle no servomotor
        pwm_set_gpio_level(SERVO, rotacao_servo*(PERIOD_PWM+1));

        return true;

    }


    void setup()

    {
        /*****************************************Configuração do
I2C*********************************************/

        // Inicializando as entradas e saídas padrões do sistema
        stdio_init_all();

        // Definindo a função dos pinos para a comunicação I2C
        gpio_set_function(I2C_SDA, GPIO_FUNC_I2C);
        gpio_set_function(I2C_SCL, GPIO_FUNC_I2C);
        gpio_pull_up(I2C_SDA);
        gpio_pull_up(I2C_SCL);

        // Inicialização da comunicação I2C com uma velocidade de 100 kHz
        i2c_init(I2C_PORT, 100*1000);


        // Criando o temporizador para realizar a coleta de dados a cada 50 ms (20
Hz)
        add_repeating_timer_ms(50, aquisicao_temporizador_callback, NULL,
&temporizador_aquisicao);

        // Inicializando o acelerômetro/giroscópio
        inicializar_mpu6050();

/*****************************************Configuração do
PWM*********************************************/

        // Variável de slice do PWM
        uint slice_servo;

        // Setando o pino de PWM
        gpio_set_function(SERVO, GPIO_FUNC_PWM);

        // Obtendo valor do slice
        slice_servo = pwm_gpio_to_slice_num(SERVO);

        // Definindo divisor de clock do PWM
```

```
        pwm_set_clkdiv(slice_servo, DIVIDER_PWM);

        // Valor do contador do PWM (período)
        pwm_set_wrap(slice_servo, PERIOD_PWM);

        // Valor inicial do pino PWM
        pwm_set_gpio_level(SERVO, 0.075*(PERIOD_PWM+1));

        // Habilitando o PWM no slice correspondente
        pwm_set_enabled(slice_servo, true);


/****************************************Configuração do
OLED*********************************************/

        //Comunicação I2C do OLED
        i2c_init(i2c1, 100*1000);
        gpio_set_function(14, GPIO_FUNC_I2C);
        gpio_set_function(15, GPIO_FUNC_I2C);
        gpio_pull_up(14);
        gpio_pull_up(15);

        // Inicializa o display para desligá-lo (caso estivesse ligado
anteriormente)
        ssd1306_t disp;
        disp.external_vcc=false;
        ssd1306_init(&disp, 128, 64, 0x3C, i2c1);
        ssd1306_poweroff(&disp);

    }



    int main()
    {
        // Realizando as configurações iniciais
        setup();

        // Declarando variável utilizada para definir o display OLED
        ssd1306_t disp;
        disp.external_vcc=false;
        ssd1306_init(&disp, 128, 64, 0x3C, i2c1);
        ssd1306_clear(&disp);

        // Verificação periódica para a realização do printf
        while (true)
        {
            // Enviando o valor lido para o monitor serial
            if(flag_printf==1)
            {
```

```
                print_velocidade = -100 + 200*normalizacao_servo ;
                printf("Leitura acelerometro (x,y,z): %.2f g, %.2f g, %.2f
g\nLeitura giroscopio (x,y,z): %.2f º/s, %.2f º/s, %.2f º/s\nAngulo phi =
%.2fº\nVelocidade do servo: %.2f%%\n\n", leitura_ac_x, leitura_ac_y, leitura_ac_z,
leitura_gyro_x, leitura_gyro_y, leitura_gyro_z, calc_phi, print_velocidade);

                // Escrevendo a mensagem desejada no display OLED:
                sprintf(info, "%.2f graus", calc_phi);
                ssd1306_draw_string(&disp, 0, 10, 1, words[0]);
                ssd1306_draw_string(&disp, 25, 25, 1, info);

                // Checando se o ângulo está acima de um limiar crítico
                if(calc_phi>90)
                {
                    ssd1306_draw_string(&disp, 0, 40, 1, words[1]);
                }

                ssd1306_show(&disp);
                ssd1306_clear(&disp);

                flag_printf = 0;
            }

        }

        return 0;
    }
```

10. Referências

AOSONG ELECTRONICS CO., LTD. ***AHT10 Technical Manual***. Guangzhou: Aosong Electronics Co., Ltd., 2018. (ASAIR® Temperature and Humidity Sensor). Disponível em: https://www.aosong.com. Acesso em: 15 out. 2025.

FISHER, Christopher J**.** ***Using an Accelerometer for Inclination Sensing.*** Norwood, MA: Analog Devices, 2010. 8 p. (Application Note AN-1057). Disponível em: https://www.analog.com. Acesso em: 15 out. 2025.

INVENSENSE INC. ***MPU-6000 and MPU-6050 Product Specification.*** Sunnyvale, CA: InvenSense Inc., 2013. 52 p. (Document Number: PS-MPU-6000A-00, Revision 3.4). Disponível em: https://www.invensense.com. Acesso em: 15 out. 2025.

ROHM SEMICONDUCTOR, LTD. *BH1750FVI* ***Ambient Light Sensor IC: Digital 16bit Serial Output Type Ambient Light Sensor IC.*** Kyoto: ROHM Co., Ltd., 2011. 17 p. (Technical Note, Rev. D). Disponível em: https://www.rohm.com. Acesso em: 15 out. 2025.

WU, Joseph. ***A Basic Guide to I2C.*** Dallas, TX: Texas Instruments Incorporated, 2022. (Application Note SBAA565). Disponível em: https://www.ti.com/lit/pdf/SBAA565. Acesso em: 15 out. 2025.